

MemPerf: Profiling Allocator-Induced Performance Slowdowns

JIN ZHOU, University of Massachusetts at Amherst, USA

SAM SILVESTRO, University of Texas at San Antonio, USA

STEVEN (JIAXUN) TANG, University of Massachusetts at Amherst, USA

HANMEI YANG, University of Massachusetts at Amherst, USA

HONGYU LIU, University of Texas at San Antonio, USA

GUANGMING ZENG, Synopsys, USA

BO WU, Colorado School of Mines, USA

CONG LIU, University of Texas at Dallas, USA

TONGPING LIU, University of Massachusetts at Amherst, USA

The memory allocator plays a key role in the performance of applications, but none of the existing profilers can pinpoint performance slowdowns caused by a memory allocator. Consequently, programmers may spend time improving application code incorrectly or unnecessarily, achieving low or no performance improvement. This paper designs the first profiler—MemPerf—to identify allocator-induced performance slowdowns without comparing against another allocator. Based on the key observation that an allocator may impact the whole life-cycle of heap objects, including the accesses (or uses) of these objects, MemPerf proposes a life-cycle based detection to identify slowdowns caused by slow memory management operations and slow accesses separately. For the prior one, MemPerf proposes a thread-aware and type-aware performance modeling to identify slow management operations. For slow memory accesses, MemPerf utilizes a top-down approach to identify all possible reasons for slow memory accesses introduced by the allocator, mainly due to cache and TLB misses, and further proposes a unified method to identify them correctly and efficiently. Based on our extensive evaluation, MemPerf reports 98% medium and large allocator-reduced slowdowns (larger than 5%) correctly without reporting any false positives. MemPerf also pinpoints multiple known and unknown design issues in widely-used allocators.

CCS Concepts: • **Software and its engineering** → **Software performance**.

Additional Key Words and Phrases: Memory Allocator, Performance Slowdowns

ACM Reference Format:

Jin Zhou, Sam Silvestro, Steven (Jiaxun) Tang, Hanmei Yang, Hongyu Liu, Guangming Zeng, Bo Wu, Cong Liu, and Tongping Liu. 2023. MemPerf: Profiling Allocator-Induced Performance Slowdowns. *Proc. ACM Program. Lang.* 7, OOPSLA2, Article 272 (October 2023), 24 pages. <https://doi.org/10.1145/3622848>

Authors' addresses: Jin Zhou, jinzhou@umass.edu, University of Massachusetts at Amherst, , USA; Sam Silvestro, Silvestro@utsa.edu, University of Texas at San Antonio, , USA; Steven (Jiaxun) Tang, jtang@umass.edu, University of Massachusetts at Amherst, , USA; Hanmei Yang, hanmeiyang@umass.edu, University of Massachusetts at Amherst, , USA; Hongyu Liu, hongyu.liu@my.utsa.edu, University of Texas at San Antonio, , USA; Guangming Zeng, zenggm@gmail.com, Synopsys, , USA; Bo Wu, bwu@mines.edu, Colorado School of Mines, , USA; Cong Liu, cong@utdallas.edu, University of Texas at Dallas, , USA; Tongping Liu, tongping@umass.edu, University of Massachusetts at Amherst, , USA.



This work is licensed under a Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International License.

© 2023 Copyright held by the owner/author(s).

2475-1421/2023/10-ART272

<https://doi.org/10.1145/3622848>

1 INTRODUCTION

Memory allocator is known to be a key system component that may greatly impact the performance of applications. Due to this reason, a rich set of memory allocators [Aigner et al. 2015; Berger et al. 2000; Evans 2016; Ghemawat and Menage 2007; Kuszmaul 2015; Leijen 2020; Rogozhin 2014] has been proposed in the past. These allocators have various design targets and trade-offs, thus yielding different performance impacts on different applications. We ran a range of widely-used applications with multiple representative allocators, including two Linux allocators (glibc-2.28 and glibc-2.21), jemalloc [Evans 2016], TCMalloc [Ghemawat and Menage 2007], and Hoard [Berger et al. 2000]. Figure 1 shows the normalized runtimes of all applications with allocator-induced slowdowns, where a lower bar indicates a better performance. We have the following observations: (1) the memory allocator may actually introduce orders of magnitude slowdowns, such as TCMalloc on cache-thrash, highlighting the importance of identifying potential allocator-induced slowdowns. (2) None of the allocators always performs the best across all evaluated applications, indicating the intricate interactions between allocators and application characteristics.

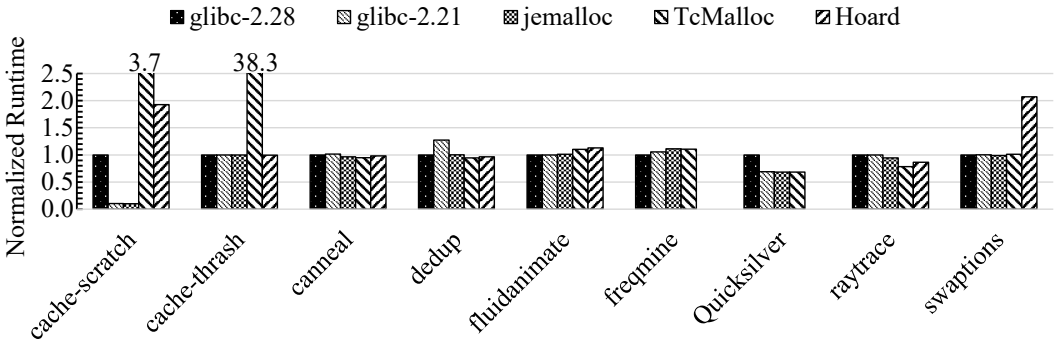


Fig. 1. Performance difference with different allocators, where all data is normalized to that of glibc-2.28.

Existing allocator profilers [Free Software Foundation, Inc. 2015; Ghemawat 2005; Lee et al. 2000; LUPASC and Popoiu 2014; Zorn and Hilfinger 1988] mainly focus on how an application uses the memory, but not allocator-induced performance slowdowns. General-purpose profilers, such as gprof [Graham et al. 1982], perf [Erastian et al. 2015], and Coz [Curtsinger and Berger 2015], focus mainly on applications, but cannot directly identify performance slowdowns caused by memory allocators. For instance, for cache-thrash in Figure 1, they at most pinpoint the few lines of application code with a high percentage of running time or cache misses, yet leaving programmers to figure out the real root cause.

People may think it unnecessary to design an allocator profiler as they could always trial different allocators and different versions, and choose the one with the best performance. However, the trials may waste unnecessary effort, if the current allocator introduces no slowdowns. Also, it is impossible to know when to stop the search for a better allocator, since none of them has the best performance for all applications as shown in Figure 1.

Further, certain applications may have very specific requirements on the allocator, preventing the change of the allocator. For instance, Firefox requires its allocator to segment different allocation types (JS strings, JS arrays, etc.) into different partitions for security purpose [Copeard 2017]. Therefore, an allocator profiler that can clearly report the potential/no slowdown of the current allocator will save the unnecessary effort of searching for a better allocator. Based on the report, users may improve the performance in the following ways, such as padding the related data

structure, altering the allocations of the application, changing the configuration flag of the allocator, or switching to a new allocator. In this paper, we propose **the first memory allocator Profiler—MemPerf**—to achieve such targets.

The proposed work does not require running or comparing multiple allocators. Intuitively, we could measure the runtimes of memory allocations and deallocations to identify an allocator’s slowdown. However, slow allocations/deallocations alone cannot explain why TCMalloc introduces around $37\times$ slowdown for cache-thrash (in Figure 1), since TCMalloc has the similar runtimes for its allocations/deallocations as other allocators. *Our key observation is that an allocator will further impact the whole life-cycle of heap objects, including the accesses (or uses) of these objects.* Therefore, we propose a **life-cycle based detection** that measures the slowdowns of (1) management operations (allocations/deallocations) and (2) memory accesses **separately**, as further discussed below.

To measure slow management operations, MemPerf proposes a novel **thread-aware and type-aware performance modeling** approach. Intuitively, MemPerf can compute the new runtime by replacing the collected runtime of the current allocator with those of an oracle allocator: if the computed runtime is shorter than the original one, the current allocator introduces a negative performance impact; otherwise, it introduces no slowdown on memory management operations. However, this method faces multiple *accuracy challenges* related to the choice of the oracle allocator, the difference of operations and applications, and multithreaded issues. MemPerf deals with all these challenges as further discussed in Section 2.1.

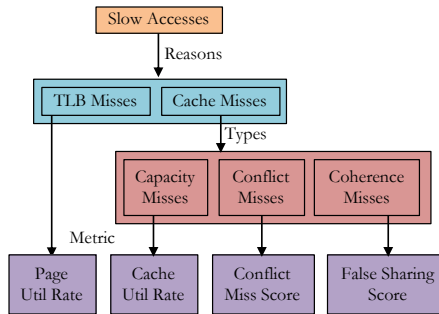


Fig. 2. Top-down approach to develop metrics. TLB misses and cache misses serve as specific breakdowns of slow accesses. Each of these breakdowns is further refined and assigned specific metrics as shown.

To measure allocator-induced slow memory accesses, MemPerf takes a **top-down approach** to develop a range of metrics, and further proposes a **unified method** to measure these metrics efficiently. It is infeasible to measure slow accesses directly without new hardware support. We observe that allocator-induced slow accesses can be caused by TLB misses and cache misses. Note that page faults are closely correlated with TLB misses, thus we focus on metrics for the latter and cache misses. For instance, the allocator could position multiple objects that are frequently accessed within the same cache set, thereby introducing cache conflict misses (called “allocator-induced conflict misses” in the subsequent sections of this paper) and slowing down the data accesses. Based on this observation, we further focus on the two types of bottlenecks caused by the allocator. For instance, we should differentiate cache misses caused by accessing global variables, or applications themselves. On top of that, we further develop a metric for each potential type of TLB misses and cache misses, as shown in Figure 2. Based on the shared properties of these metrics, we further propose the employment of *precise address sampling* to measure these metrics correctly and efficiently, by taking full advantage of all sampled information as discussed in Section 2.2.

Extensive experiments show that MemPerf can identify 98% of allocator-induced more than 5% slowdowns correctly, but without reporting any non-existent slowdowns (or false positives). These slowdowns can be caused by either slow management operations or slow accesses. MemPerf further pinpoints multiple known and **new** design issues of multiple well-known allocators. Overall, MemPerf only imposes 7.5% performance overhead (GEOMEAN) on average, minimizing its interference to the target program and making it applicable even to the real deployment environment. To use MemPerf, there is no need to change the application and the underlying OS. In principle, MemPerf does not need to modify the allocator, when an allocator is a separate library that does not use customized system calls or synchronizations, or users do not want to know the specific reason for slow management operations. For instance, when an allocator uses custom system calls, e.g., the default Linux library, MemPerf cannot intercept the system calls using the preloading mechanism. But the profiler can still identify slow management operations and slow memory accesses. In summary, this paper makes the following analytical and methodological contributions:

- It is the first work that systematically analyzes various reasons for allocator-induced slowdown, and proposes a “life-cycle based detection” approach to measure all slowdowns.
- It proposes thread-aware and type-aware performance modeling to measure slow memory management operations.
- It proposes a top-down approach to analyze reasons for slow memory accesses, and further proposes a range of metrics and practical implementations to measure allocator-induced TLB and cache misses correctly and efficiently.
- Extensive evaluations confirmed MemPerf’s effectiveness and efficiency.

2 BASIC IDEAS OF MEMPERF

This section discusses the basic ideas of measuring slow management operations and slow accesses.

2.1 Identifying Slow Management Operations

We propose a **thread-aware and type-aware performance modeling** to predict the slowdown caused by memory management operations. MemPerf will report the level (low, medium, and large) of slowdown introduced by management operations of the current allocator, without comparing it with another allocator. The basic idea is to replace the cycles of management operations of the current allocator with the cycles of an *oracle allocator*, and then compute the new runtime after the replacement. In the modeling, we assume that the runtime of memory accesses will remain the same, but actual slow accesses can be measured with a different method. Let us use a simple example: a program runs 50,000 cycles, spending 20,000 cycles for its 200 memory allocations/deallocations under the current allocator, and 30,000 cycles for other operations; If each allocation of the oracle one requires only 50 cycles, then the allocation time will be $200 \times 50 = 10,000$ cycles for the oracle allocator. Therefore, management operations will be 10,000 fewer cycles, and the performance will be improved by $10,000/50,000 = 20\%$ when switching to a faster allocator.

Although the replacement-based method seems intuitive, MemPerf should handle multiple accuracy challenges as mentioned in Section 1. (1) *Choose the oracle allocator*: The oracle allocator should be the one with the fastest memory management operations, but not the one with the best performance. Based on this, TCMalloc is chosen as the oracle allocator, as it has the fastest management operations, as evaluated in Section 4.2.1. (2) *Variance of operations*: Different types of management operations usually have significantly different performance, since an allocator handles them in different execution paths. For instance, most of the existing allocators have the “small/big object” design: for small objects with sizes smaller than a pre-defined threshold, an allocator will satisfy the requests from the local cache; for big objects, the allocator may directly invoke `mmap()`

or `sbrk()` to obtain the requested chunk from the OS. Thus, allocations of small and big objects usually have different performance and should be considered separately. Based on our evaluation, the average runtime of `glibc-2.28`'s new allocations is around 484 cycles (in serial phase), which is $5.3\times$ of its re-used allocations. MemPerf will track the type of operations, such as allocations of small and big objects, allocations of newly-allocated and re-allocated objects, or allocations in serial and parallel phases, and replace them with the runtime of the corresponding types. (3) *Variance of applications*: The cycles of memory management operations for the same allocator can be changed dramatically for different applications, due to the scheduling effect or other reasons. To avoid application-specific impacts, we will utilize the average data of a large number of applications as the oracle data for different memory management operations.

In fact, the biggest challenge is caused by multithreading, which is ubiquitous due to the widely-used multi-core and many-core machines. In general, MemPerf treats the runtime of any application as the summary of its serial phases and parallel phases. Note that a single-threaded application is just a special type with only one serial phase. For multithreaded applications, the key point is to compute the runtimes of parallel phases correctly. MemPerf abstracts multithreaded applications as shown in Figure 3. For this figure, since the parallel phase has thread T1, T2, and T3, MemPerf tracks the runtimes and numbers of memory management operations in these threads. Then it computes the new runtimes of T1, T2, and T3 by replacing them with the oracle allocator's runtimes. After that, the thread with the longest runtime will be the new runtime for the corresponding parallel phase.

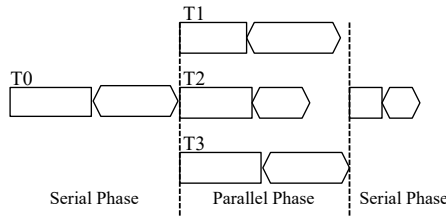


Fig. 3. Performance modeling for multithreaded applications, where rectangles represent the cycles of management operations and hexagons are the cycles of other operations.

The simplified model of Figure 3 works very well for most applications of fork-join models, but not for applications with inter-dependence between threads. For instance, in `dedup`, the longest thread of the parallel phase is simply waiting on other threads. Although the cycles of other threads are significantly reduced, the longest thread is not changed much due to very few operations on this thread. For such a case, the runtime of the parallel phase at most could be reduced to that of the second-longest thread. That is, the length of second-longest thread provides an upper bound for the possible performance improvement for the specific parallel phase.

2.2 Identifying Slow Memory Accesses

Based on Section 1, MemPerf takes a top-down approach to develop a set of metrics related to TLB/cache misses. These metrics are called *application friendliness factors*, which measure whether the allocator is cache or TLB friendly to the specific application. These metrics are discussed as follows.

Page Utilization Rate (PUR): Page utilization rate, indicating the percentage of pages holding actually heap objects, will be utilized to measure the allocator's impact related to TLB misses. For the same application, an allocator with a lower page utilization rate potentially causes more

TLB misses, and therefore worse performance. MemPerf does not utilize TLB misses directly to evaluate an allocator's impact, as TLB misses can be caused by global variables or text segments. Instead, PUR only tracks the utilization rate of pages holding heap objects, which is better suited for evaluating an allocator's performance impact due to TLB misses.

However, cache misses are more complicated, due to multiple types of cache misses. Among them, compulsory misses are considered to be mandatory and unavoidable, which are omitted by MemPerf. Capacity misses may occur if the working set of a program exceeds the capacity of the cache, conflict misses will be introduced when more than N cache lines are mapping to the same set in N -way associative cache, and coherency misses may occur when multiple threads are accessing the same cache line simultaneously. MemPerf makes the following observations: *an allocator may affect capacity, conflict, and coherency misses*. Therefore, MemPerf provides different metrics to evaluate each type of cache misses separately.

An alternative is to actually predict possible costs caused by the allocator-induced cache/TLB misses. However, in order to achieve this target, we may need to record all synchronizations, memory operations, and system calls, and then infer the possible impact caused by these cache misses. This approach will have two issues: (1) we may require to record a large number of events, which could possibly impose significant performance overhead. Unfortunately, the increased runtime overhead will also significantly impact the recording accuracy, affecting the accuracy of prediction. (2) Further, performance modeling of cache behavior is still an unsolved issue, since the performance of an application can be affected by many complicated factors, e.g, the variances of cache loading speed and data dependencies. Based on these considerations, MemPerf decided to report some metrics indicating serious cache misses.

Cache Utilization Rate (CUR): Cache utilization rate, the percentage of cache lines holding the actual heap objects, is used to evaluate cache capacity misses. For the same application, an allocator with higher internal fragmentation may have a lower cache utilization rate, or vice versa. An allocator with a higher cache utilization rate will have a better performance. Similar to PUR, CUR only tracks cache lines related to heap objects, skipping those ones related to global variables and text segments. Therefore, it is better to evaluate the impact of the allocators than the number/percentage of cache misses.

Conflict Miss Score (CMS): An allocator introduces conflict misses, when multiple allocated objects are unfortunately mapped to the same cache set. MemPerf proposes conflict miss score (CMS) to measure the seriousness of allocator-induced conflict misses. Intuitively, one cache set s with a higher cache miss rate will have a higher performance impact. Based on this intuition, CMS_s , indicating the severeness of allocator-induced conflict misses on the specific set s , is defined as follows:

$$CMS_s = \frac{Miss_Ratio_s}{Miss_Ratio_{avg}} \quad (1)$$

In Equation 1, $Miss_Ratio_s$ is the cache miss rate of cache set s , and $Miss_Ratio_{avg}$ indicates the average miss ratio for all cache sets. In particular, for a machine with 64 cache sets, the average cache miss ratio of every cache set $Miss_Ratio_{avg}$ should be 1.5625% (i.e., $1/64$). Based on the above formula, CMS of an application can be computed as follows.

$$CMS = \sum_{i=0}^n (CMS_{s_i} \times c^i) \quad (2)$$

In Equation 2, c is a reduction coefficient, introduced to adjust the CMS of each cache set in relation to its ranking by miss ratio. With a default value of 0.5, c ensures that the cache set with the highest miss ratio receives the full CMS score, while each subsequent set's score is reduced

by half, reflecting the relative impact of each set on the overall performance. For instance, if two cache sets have the *CMS* scores to be 16 and 10 separately, then the application's *CMS* will be $16 + 0.5 \times 10 = 21$. *The rationale of using the reduction coefficient is that allocator-induced conflict misses are more likely to occur in a few cache sets.* Let's illustrate this using two simple examples: (1) if the memory accesses and cache misses of each cache line are perfectly balanced, then the *CMS* of each cache line will be 1, and the overall *CMS* will be 64 for 64 cache sets. (2) There is one cache line with its *CMS* score to be 64. Clearly, the second case is more likely caused by the allocator, as significant cache misses are related to the same cache line. Instead, the allocator cannot be improved for the first case. Therefore, MemPerf utilizes Equation 2 to filter out conflict misses induced by the allocator. Our evaluation results shown in Table 4 also confirm that this mechanism can filter out allocator-induced conflict misses, as the fixes of these found issues improve the performance between 6% to 55%.

False Sharing Score: When an allocator allocates different objects in the same cache line to different threads, and these threads are accessing these objects concurrently, the allocator will introduce a false sharing problem. MemPerf proposes the **False Sharing Score (FSS)** to evaluate the seriousness of false sharing. A cache line's FSS_c is computed as follows, using the cache miss ratio.

$$FSS_c = Miss_Ratio_c * 100 \quad (3)$$

Here, the percentage is multiplied by 100 to obtain a value between 0 and 100. Since multiple cache lines of an application may have false sharing, the *FSS* of an application can be computed as follows. c is a reduction coefficient, introduced to adjust the *FSS* of each cache line in relation to its ranking by miss ratio. With a default value of 0.5, c ensures that the cache line with the highest miss ratio receives the full *FSS* score, while each subsequent line's score is reduced by half, reflecting the relative impact of each line on the overall performance.

$$FSS = \sum_{i=0}^n (FSS_{c_i} \times c^i) \quad (4)$$

2.2.1 Basic Idea of Measuring Metrics. Based on our observation, all the above-mentioned metrics are related to memory accesses and cache misses (e.g., *CMS* and *FSS*) separately. In particular, memory accesses can be utilized to measure page/cache utilization rate. For instance, upon each memory access, we could check the utilization rate of the related cache line and page, and then compute the average utilization rate correspondingly. For instance, if the first access's page utilization rate is 50% and then the second access's page utilization rate is 70%, then the overall page utilization rate will be $(50\% + 70\%)/2 = 60\%$. The cache utilization rate can be computed similarly. However, it is very expensive to employ compiler-based instrumentation [Serebryany et al. 2012] or binary instrumentation [Luk et al. 2005] to trace memory accesses, which may introduce over $2\times$ performance overhead. Further, they cannot report cache misses directly that are required to compute *CMS* and *FSS*, which may require some complicated simulation.

Instead, MemPerf further proposes to employ *precise address sampling* to measure these metrics efficiently and correctly. Precise address sampling provides the detailed information of each sampled memory access [Liu and Mellor-Crummey 2013; Liu et al. 2014; Sheng et al. 2011], such as its memory address, its related instruction, and its hit information (indicating a cache miss or not) that is often omitted by existing work [Chabbi et al. 2018; Liu and Liu 2016]. Instead, MemPerf will utilize the hit information to identify cache misses.

To reduce the interference with the original allocator, MemPerf uses a separate allocator for its internal objects, which is from a separate address range from memory objects of applications. This design allows MemPerf to easily differentiate memory accesses of the allocator from those of

the application, by checking which range the sampled address is located in. That is, MemPerf will minimize the impact on the allocator's performance, due to the use of a different allocator. Further, MemPerf separates conflict and coherency misses of applications from those of allocators, and only reports allocator-induced scores.

3 IMPLEMENTATION

MemPerf is implemented as a library that can be linked with applications dynamically, without changing the allocator, the application, and the OS. There is no need of recompiling applications as well.

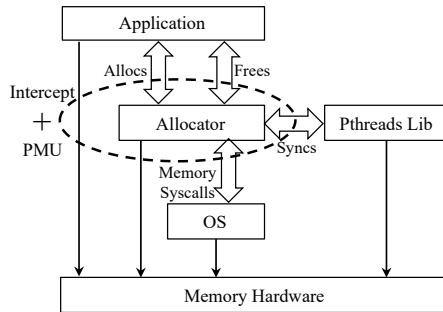


Fig. 4. Overview of MemPerf.

Overall, MemPerf intercepts some functions and performs precise address sampling as shown in Figure 4. In order to model slowdowns of slow management operations, MemPerf intercepts all memory allocations and deallocations in order to collect their runtimes. In order to pinpoint the specific reasons for slow memory management operations, MemPerf further intercepts synchronizations, and memory-related system calls. MemPerf employs precise address sampling of Performance Monitoring Units (PMU) to collect the information of memory accesses in order to identify slow memory accesses. The details of its implementations are discussed as follows.

3.1 Identifying Slow Management Operations

For this purpose, MemPerf intercepts all memory management operations in order to track the runtime of each memory operation. MemPerf employs the RDTSCP instruction to track the runtime because of its low overhead and fine-grained granularity (e.g. in the cycle-level) [Intel Corporation 1997; Yang 2015]. It obtains the timestamp before and after each management operation, and then utilizes the difference as the runtime for this operation.

As mentioned in Section 2.1, it is important for MemPerf to differentiate different types of management operations. Basically, memory allocations can be further classified into small/big based on the size, new/re-used allocation in the serial/parallel phase. Similarly, deallocations can be further classified into small/big deallocation in the serial/parallel phase. Some allocators, e.g., Hoard, even have medium objects. MemPerf differentiates the size of an allocation as described in Section 3.3. To differentiate re-used allocations from new ones, MemPerf utilizes a hash table to track all allocated objects: an object (with address and size information) is inserted into the hash table at its allocation time, and will be kept in the table even after its deallocation. Upon every allocation, MemPerf checks whether an object is already in the hash table by the address. If yes, then this allocation is a re-used allocation. Otherwise, it is a new allocation. This hash table is used to check the size of an object during the deallocations. MemPerf differentiates between operations

executed in serial and parallel phases. This distinction is made by tracking the number of live threads. More specifically, MemPerf intercepts thread creation functions and uses the active thread count to determine the current phase: a single live thread indicates a serial phase, whereas multiple threads indicates a parallel phase.

MemPerf minimizes the possible interference of the execution by tracking the runtime and the number of each type of operations separately in per-thread local storage, which avoids the possible lock contention. After getting the runtime information, MemPerf detects slow management operations by replacing the runtime of different types with the corresponding ones of the oracle allocator as described in Section 2.1: if the computed runtime is smaller than the actual runtime, the current allocator introduces a negative performance impact. Otherwise, there is no impact.

Understanding user and kernel space lock contention: Many slow memory management operations are caused by user-space and kernel-space lock contention within memory management operations. To provide more information that guides the fix, MemPerf will report whether an allocator introduces user-space and kernel-space contention, and which lock is the culprit for the user-space contention, aiming to provide more information. Note that MemPerf only focuses on lock contention inside management operations, not all lock contentions that are different from general synchronization profilers [Alam et al. 2017; Yu and Pradel 2016; Zhou et al. 2018].

For user-space contention, MemPerf intercepts standard mutex locks (e.g., `mutex->__data.__lock`). If an allocator is using different types of synchronization, such as Hoard [Berger et al. 2000], either its synchronizations should be replaced with standard synchronizations or MemPerf cannot report the details of the contention inside. That is, if users do not want to know user-space contention, then MemPerf does not need the change of all allocators at all. For each type of management operations (e.g., `malloc` and `free`), MemPerf collects the numbers and runtimes of lock acquisitions, and the numbers and percentage of lock contention inside. For kernel-space contention, MemPerf focuses mostly on memory-related system calls inside memory management operations, such as `mmap`, `madvise`, or `munmap`. These system calls invoke a per-process lock inside the kernel, which will also conflict with the handler of page faults. The average runtime of these system calls can be utilized to determine the kernel-level contention, without the requirement of changing the OS. That is, if the runtime of each system call is larger than the normal value, then the kernel is suffering from kernel contention. The kernel-space contention is found to be the slowdown reason for `dedup` with `glibc-2.21`, introducing 20% slowdown. To reduce the profiling overhead, MemPerf only samples locks and relevant system calls with a sampling rate of 1%, without compromising the effectiveness.

3.2 Identifying Slow Memory Accesses

As discussed in Section 2.2, MemPerf measures a range of application friendliness factors, such as page/cache utilization rate, conflict miss score, and false sharing score. MemPerf relies on hardware Performance Monitoring Units (PMU) to sample memory accesses. For each sampled memory access, MemPerf is able to obtain the address, cache miss information, and thread index information, which will be used to compute these factors. In particular, we will use the address to update the page/cache utilization rate, and use the cache miss and the thread index to update CMS/FSS. Currently, the sampling rate is set to sample one load/store access out of 500,000 accesses, in order to balance the accuracy and performance overhead. The impacts of different sampling rates are further evaluated in Section 4.5. The detailed implementations are discussed as follows.

Page/Cache Utilization Rate: MemPerf reports the average page/cache utilization rate for all sampled accesses. Basically, MemPerf maintains the allocated bytes of each cache line and each page. For instance, if a cache line (64 bytes) only has one valid object with 24 bytes, then the cache utilization rate is $24/64 = 37.5\%$. MemPerf intercepts all allocations and deallocations in order to

update the allocated bytes of each cache line. Upon each sampled access, MemPerf collects the page/cache utilization rates related to this access, and updates them correspondingly. Therefore, MemPerf requires to access the allocated bytes of each cache line and each page frequently.

In order to reduce the overhead of updating and checking, MemPerf adopts the shadow memory technique [Serebryany et al. 2012] to store the allocated bytes of each cache line and each page: one continuous memory block is utilized to hold the information of cache lines, and one for pages. Therefore, every cache line or page of the heap is mapped to a shadow address that can be computed with few operations. The shadow memory may introduce a higher memory consumption.

Conflict Miss Score (CMS): MemPerf attributes cache misses to each cache set, where each set maintains the number of cache misses, and an array of cache lines with cache misses. As described in Section 3.1, MemPerf utilizes a hash table to track the addresses, allocation callsites, and sizes of all allocated objects. After obtaining the sampled memory access, MemPerf will update the related information in the hash table. When an object is deallocated, MemPerf gets its address range via the hash table, checks whether the object is located in the cache lines with cache misses, and maps the object's information to the corresponding cache line(s). If a few sets have a larger number of cache misses than others, and their related cache lines are landing on *multiple heap objects*, then there exist allocator-induced cache conflict misses. MemPerf will report all cache sets with cache miss rates larger than 5%, and compute the CMS of the application. If none of these sets satisfies, MemPerf reports 0 on CMS. MemPerf could report the information (e.g., allocation site and instruction) of all related objects, which could help guide the problem fix. For instance, users could insert some bogus allocations or change the size of allocations to avoid allocator-induced conflict misses, as further discussed in Section 4.3.

False Sharing Score (FSS): MemPerf utilizes Equation 4 to measure the allocator-induced false sharing score of an application. As an application itself may also have some false sharing issues, MemPerf could report the overall FSS and the allocator-induced FSS. As the first access of a cache line may actually introduce compulsory miss, MemPerf first records the number of misses of each cache line, and then record more detailed information once the number of cache misses is larger than a threshold (e.g., 5 misses). Therefore, this mechanism could exclude sporadic cache misses. MemPerf also reports the information related to cache lines, such as the related object's callsite. Further, MemPerf is able to differentiate false sharing from true sharing, by recording the thread access information for cache lines with cache misses. If different threads are landing on the same words, then it is a true sharing problem. Note that because MemPerf utilizes the hit information of each memory access, it does not need to simulate cache invalidations as existing work [Liu and Berger 2011; Liu and Liu 2016].

3.3 Adapting to Different Allocators

MemPerf is designed as a general profiler for different allocators. It needs to classify small, medium, or big objects, since they have different execution paths as discussed in Section 2.1. To achieve this, programmers could provide a simple execution file that lists the thresholds of different types of objects inside the allocator (e.g., small, medium, and big). Alternatively, MemPerf also provides a pre-run program to automatically determine the thresholds for a given allocator. We do not consider it as a contribution of MemPerf, but as a way to help provide the configuration file. The threshold for big objects is detected by checking whether there is an explicit `mmap` or `sbrk` system call upon the allocation request, since most allocators utilize either of the system calls to satisfy the allocation of a big object. However, this threshold still requires manual confirmation.

3.4 Signal-Triggered On-Demand Report

Normally, MemPerf reports the results at the end of the execution. However, this design is not suitable for server applications that are running forever. Therefore, MemPerf also supports signal-triggered reporting by installing a signal handler. In the signal handler, MemPerf further propagates this signal to all alive threads. Whenever receiving the signal, every thread immediately merges its local data to the global area under the protection of a spinlock. After every thread confirmed the completion of the data merge, MemPerf reports the result in the same way as that at the end of the execution.

4 EXPERIMENTAL EVALUATION

4.1 Experimental Environment

Hardware/Software Platform: Experiments were performed on a two-processor machine, both processors are Intel(R) Xeon(R) Gold 6230 with 20 cores. We only utilize 16 hardware cores in one node and disable the rest cores to exclude the NUMA effect. This machine has 256GB of main memory, 64KB L1 cache, and 1MB of L2 cache. The OS is Ubuntu 18.04.3 LTS, installed with kernel version Linux-5.3.0-40. The compiler is GCC-7.5.0, with `-O2` and `-g` flags.

Evaluated Memory Allocators: We evaluated MemPerf with five widely used allocators, including two Linux allocators (glibc-2.28 and glibc-2.21), TCMalloc-4.5.3 [Ghemawat and Menage 2007], jemalloc [Evans 2016], and Hoard [Berger et al. 2000]. Among them, TCMalloc is developed and widely used inside Google, and jemalloc is designed by Facebook. glibc-2.28 is the default allocator in our current machine. MemPerf *does not need the change of allocators, (1) if allocators use standard synchronizations and system calls, or (2) if users do not want to know the details related user/kernel contention*. In our evaluation, we recompiled two glibc allocators separately as stand-alone libraries, and replaced Hoard's internal synchronizations with POSIX spinlocks. We have confirmed that these changes did not introduce observable performance impacts, with less than 1% performance impact.

Evaluated Applications: We evaluated the PARSEC applications [Bienia and Li 2009], two stress tests cache-thrash and cache-scratch from Hoard [Berger et al. 2000], multiple real applications including aget, Apache, memcached, pbzip2 and pfscan, and AMG, Nekbone, PENNANT, and Quicksilver from CORAL-2 benchmarks [Laboratory 2018]. We used the native inputs for PARSEC applications. For memcached, we ran `memtier_benchmark` in the protocol of `memcache_binary`. For Apache, we used the `ab` script to send 25,000 requests. For aget, we tested it by downloading a 30MB file in a local network. pfscan searched a keyword in a 500MB file. For Pbzip2, we tested it by compressing 10 files with 30MB each. For applications in CORAL-2 benchmarks, we used OpenMP [ARB 2022] to run 16 threads. For AMG, we also used MPI [Project 2022] to run a single process. We tested AMG by solving the first given problem, and set the problem size $200 \times 200 \times 200$ and the processor topology $1 \times 1 \times 1$. For Nekbone, we set the problem size for each processor $256 \times 256 \times 1$, and used default values for other arguments. For PENNANT, we tested it by solving the `nohpoly` problem. For Quicksilver, we set the number of particles to 5,000,000 and used default values for other arguments. All applications were compiled by GCC-7.5.0 with `-O2`, which is the default compiler in our machine.

Comparison with Existing Work: We compare MemPerf with perf [Eranian et al. 2015], CCProf [Roy et al. 2018], and Feather [Chabbi et al. 2018] on both effectiveness and efficiency. Among them, CCProf detects conflict misses and Feather detects false sharing.

4.2 Effectiveness Evaluation

We evaluated whether MemPerf could identify the allocator's slow management operations and slow memory accesses below.

Table 1. The average cycles of memory management operations in PARSEC applications [Bienia and Li 2009]. Bold numbers are the fastest ones in each type of operations. Among the evaluated allocators, TCMalloc has the best performance in 7 of the 10 types of memory management operations, which is the reason why it is chosen as the oracle allocator.

Allocator	Small/Big Object Threshold	Small Objects						Big Objects			
		New Alloc		Re-Use Alloc		Dealloc		Alloc		Dealloc	
		Serial	Para	Serial	Para	Serial	Para	Serial	Para	Serial	Para
glibc-2.28	132240	484	1103	91	242	156	317	7622	2785	1962511	26646
glibc-2.21	132880	394	1520	300	367	146	539	6937	2693	1912669	44060
jemalloc	65536	153	375	79	127	167	233	7662	1892	4260	2179
TCMalloc	188424	149	304	69	123	111	103	7120	1308	23251	1937
Hoard	8192	292	702	79	1177	189	1406	18126	1517	199770	608

4.2.1 Identifying Slow Management Operations. Selecting Oracle Allocator: In order to identify slow management operations, we will use an oracle allocator for performance modeling. We ran PARSEC [Bienia and Li 2009] with different allocators, and compared their performance of memory management operations, as shown in Table 1. Among the evaluated allocators, TCMalloc shows the best performance in 7 of 10 types of memory management operations, while the other 3 types of operations are rarely invoked (less than 0.01%). Based on the evaluated results, MemPerf employs TCMalloc as the oracle allocator using the cycles in Table 1, as TCMalloc is the allocator with the fastest management operations. Note that TCMalloc is not the fastest allocator, as it introduces false sharing for some applications, as shown in Figure 1. An alternative is to always choose the best number out of these evaluated allocators. However, we expect that it should not introduce much difference, as the other 3 types of operations are very rare in the evaluated applications (with less than 0.01% based on our evaluation).

Comparing MemPerf's effectiveness with the other two methods: In order to show MemPerf's effectiveness in detecting slow management operations caused by the allocator, we compare MemPerf with two other methods.

One is a naive method, marked as "NV" in Table 2. For this method, we do not consider the types of memory management operations. That is, we just use the average cycles of memory allocation and deallocation, obtaining from TCMalloc for all PARSEC applications. As seen in Table 1, different types of memory management operations may have up to 336× difference for TCMalloc. Another method is to use the ideal allocator that takes no overhead for any memory management operations. The results of the ideal allocator is shown as "ID" in Table 2.

We acknowledge that it is extremely challenging to predict the exact performance impact of slow memory management operations of memory allocators, due to the interactions among multiple threads inside the same application [Curtsinger and Berger 2015]. Further, it is unnecessary for normal users to know the exact number. Instead, a user may decide to change or improve the allocator if the performance impact is larger than a threshold, e.g., 5%. Based on these consideration, we divide allocator-induced slowdown into three levels: Small (< 5%), Medium ($\geq 5\%$ and < 15%), and Large ($\geq 15\%$), where shown as "S", "M", and "L" correspondingly in "Impact" columns in Table 2. Note that the slowdown is obtained when comparing with the fastest evaluated allocator. For instance, fluidanimate runs the fastest when using glibc-2.28, but it runs more than 5% slower when using Hoard and TCMalloc. That is the reason why "Impact" is marked as "S" for glibc-2.28,

Table 2. Effectiveness of MemPerf (labeled as “MP”), the naive mechanism (labeled as “NV”), and the ideal allocator (labeled as “ID”) on identifying slow management operations for all applications in Figure 1. A correct identification is labeled as $\checkmark$, an incorrect identification is labeled as $\times$, and those ones with Δ are caused by slow memory accesses.

Application	glibc-2.28				glibc-2.21				jemalloc				TCMalloc				Hoard			
	Impact	NV	ID	MP	Impact	NV	ID	MP	Impact	NV	ID	MP	Impact	NV	ID	MP	Impact	NV	ID	MP
cache-scratch	L	Δ	Δ	Δ	S	$\checkmark$	$\checkmark$	$\checkmark$	S	$\checkmark$	$\checkmark$	$\checkmark$	L	Δ	Δ	Δ	L	Δ	Δ	Δ
cache-thrash	S	$\checkmark$	$\checkmark$	$\checkmark$	S	$\checkmark$	$\checkmark$	$\checkmark$	S	$\checkmark$	$\checkmark$	$\checkmark$	L	Δ	Δ	Δ	S	$\checkmark$	$\checkmark$	$\checkmark$
canneal	M	$\times$	$\checkmark$	$\checkmark$	M	$\times$	$\checkmark$	$\checkmark$	S	$\checkmark$	$\times$	$\checkmark$	S	$\checkmark$	$\checkmark$	$\checkmark$	S	$\checkmark$	$\times$	$\checkmark$
dedup	M	$\times$	$\checkmark$	$\checkmark$	L	$\times$	$\checkmark$	$\checkmark$	M	$\times$	$\checkmark$	$\checkmark$	S	$\checkmark$	$\checkmark$	$\checkmark$	S	$\checkmark$	$\checkmark$	$\checkmark$
fluidanimate	S	$\checkmark$	$\checkmark$	$\checkmark$	S	$\checkmark$	$\checkmark$	$\checkmark$	S	$\checkmark$	$\checkmark$	$\checkmark$	M	$\times$	$\times$	$\times$	M	$\times$	$\times$	$\times$
freqmine	S	$\checkmark$	$\checkmark$	$\checkmark$	M	Δ	Δ	Δ	M	Δ	Δ	Δ	M	Δ	Δ	$\checkmark$	N/A	N/A	N/A	N/A
Quicksilver	L	Δ	Δ	Δ	S	$\checkmark$	$\checkmark$	$\checkmark$	S	$\checkmark$	$\checkmark$	$\checkmark$	S	$\checkmark$	$\checkmark$	$\checkmark$	N/A	N/A	N/A	N/A
raytrace	L	Δ	Δ	Δ	L	Δ	Δ	Δ	L	Δ	Δ	Δ	S	$\checkmark$	$\checkmark$	$\checkmark$	M	Δ	Δ	Δ
swaptions	S	$\checkmark$	$\checkmark$	$\checkmark$	S	$\checkmark$	$\checkmark$	$\checkmark$	S	$\checkmark$	$\checkmark$	$\checkmark$	S	$\checkmark$	$\checkmark$	$\checkmark$	L	$\checkmark$	$\checkmark$	$\checkmark$

but marked as “M” for Hoard and TCMalloc in Table 2. As other allocators run less than 5% slower than glibc-2.28 here, that is the reason why they are marked as “S” in the table. For simplicity, Table 2 only shows the results of applications with at least one allocator-induced slowdown. The shown results also take the “boundary effect” into consideration, where the predicted difference of less than 5% will be considered a correct report. For instance, if an allocator is running 7% slower (medium impact), but MemPerf reports around 4% slower (small impact), this case will be considered as a correct detection since $107\%/104\% = 102.8\%$.

Overall, we are using 24 applications to evaluate MemPerf’s effectiveness. As shown in Table 2, 9 out of 24 applications have obvious performance downgrade (larger than 5% slowdown when comparing with the fastest allocator). For the other 15 applications, there is no observable performance difference (less than 5%) when running with different allocators. For these applications, MemPerf does not report a medium or large impact for these allocators. That is, MemPerf does not have false positives for all of these applications.

In total, there are 112 cases with five different allocators and 24 applications, excluding non-runnable executions, such as AMG, ferret, freqmine, PENNANT and Quicksilver with Hoard, and AMG, Apache and Nekbone with TCMalloc. MemPerf cannot attribute 14 cases to slow memory management operations (marked as $\times$ and Δ in Table 2). However, **this does not indicate MemPerf’s failure, but because 12 out of 14 slowdowns (marked as Δ ones) are actually caused by slow memory accesses**, as further evaluated in Section 4.2.2. Overall, MemPerf only cannot detect issues in TCMalloc and Hoard for fluidanimate. We also collected the detailed memory management cycles (with MemPerf), cache misses, and TLB misses using perf [Eranian et al. 2015] for glibc-2.28, TCMalloc, and Hoard. However, we do not observe any significant difference among these data. In fact, TCMalloc’s memory management operations are actually faster than glibc-2.28, although it is running slower with TCMalloc. Based on the reported scores, we also cannot know the slowdown reasons for these two cases.

In contrast, the naive allocator (marked as “NV” in the table) cannot predict the performance impact for five more cases, such as glibc-2.28 and glibc-2.21 for the dedup application. This shows the importance of considering the types of management operations when predicting the slow management operations. The ideal allocator (without management overhead) cannot predict the performance impact of jemalloc and Hoard for the canneal application. Therefore, these results indicate MemPerf’s effectiveness, due to its careful design, such as differentiating different types of management operations.

Allocator-induced slow memory management operations (all marked as ✓ in “MP”) in Table 2 have different root causes. Among them, Hoard introduces serious lock contention in swaptions and glibc-2.21 introduces significant kernel-contention in dedup, as further discussed in Section 4.3. Other ones in canneal and dedup are caused by slow management operations of the corresponding allocator, which can be only fixed by switching to a new allocator (as shown in Table 3).

Table 3. Approaches to fix different types of allocator-induced slowdown. “Switch” is to use another allocator, “Flag” is to change the configuration of the allocator, “Padding” is to change the size of the corresponding objects, and “Bogus” is to insert additional bogus/useless objects to prevent conflict misses. In Column “Allocator Fix” and “Application Fix”, “✓” indicates that an issue can be fixed by the corresponding method, and “✗” indicates the issue cannot be easily solved.

Category	Issue	Allocator Fix		Application Fix	
		Switch	Flag	Padding	Bogus
Slow Memory Management Operations	Slow Operations	✓	✗	✗	✗
	Kernel Contention	✓	✓	✗	✗
	Lock Contention	✓	✓	✗	✗
Slow Memory Accesses	False Sharing	✓	✗	✓	✗
	Conflict Misses	✓	✗	✓	✓

4.2.2 Detecting Slow Memory Accesses. MemPerf further evaluates application friendliness factors for all allocators on all evaluated applications. Due to the page limit, we only listed application friendliness factors, such as page/cache utilization rate, conflict miss score, and false sharing score, for executions marked as Δ in Table 2. MemPerf does not report abnormal numbers for applications without significant slowdowns.

Table 4 shows that MemPerf identifies abnormal factors for slow executions that cannot be explained by slow management operations. For cache-scratch, MemPerf reports high false sharing scores for glibc-2.28, TCMalloc, and Hoard. This explains the major reason for their slowdowns. Similarly, TCMalloc’s false sharing is the reason for its slowdown on cache-thrash. For freqmine, three allocators, glibc-2.21, jemalloc, TCMalloc, introduce some conflict misses. glibc-2.28 introduces conflict misses on Quicksilver, and multiple allocators introduce conflict misses on raytrace as shown in the “CMS” column of Table 4, where are discussed in Section 4.3. glibc-2.28 and glibc-2.21 also have lower cache and page utilization rates additionally, which help explain its higher performance degradation than jemalloc.

In summary, we have the following observations about MemPerf: (1) MemPerf successfully reports 110 out of 112 executions performance slowdowns caused by allocators by combining slow management operations and slow memory accesses, with a percentage of 98.2% but without any false positive. (2) There is no need to run other allocators to identify performance slowdowns of the current allocator. (3) CMS and FSS are good indicators for slow memory accesses, but PUR and CUR are more questionable that depend on the memory usage of applications. For instance, most allocators have low PUR or CUR on cache-scratch.

Buggy Design of Allocators: Among all issues, many of them can be attributed to the buggy design of the allocator. Here, we use “buggy” to indicate design decisions or practices that, while not causing outright system errors, can nonetheless lead to significant performance degradation. This definition also applies to all subsequent uses of “buggy” and “bug” in this paper. Based on our understanding, all false sharing issues (e.g., Hoard, TCMalloc, and glibc-2.28) can be attributed to their buggy design. For instance, different threads may obtain different objects from the same cache line, causing false sharing issues. In addition, Hoard’s medium-size heap management may introduce significant lock contention, which has been confirmed by Hoard authors, and glibc-2.21

Table 4. The allocator-induced slowdowns detected by MemPerf, perf [Eranian et al. 2015], CCProf [Roy et al. 2018], and Feather [Chabbi et al. 2018]. For applications with slow memory accesses, we also list their friendliness factors reported by MemPerf. The “Speedup After Fix” column indicates the performance improvements after fixing the issues. In the last four columns, “✓” indicates that the tool correctly detects and reports the issue, “✗” indicates the tool may detect the issue but does not provide enough information to guide the bug fix, “✗” indicates a fail detection, and “N/A” indicates that the corresponding application crashes when running with the tool. Column “New” indicates whether the issue is firstly discovered by MemPerf.

Category	Application	Allocator	Issue	Speedup After Fix	Friendliness Factors				MemPerf	perf	CCProf	Feather	New
					PUR	CUR	CMS	FSS					
Slow Memory Management Operations	canneal	glibc-2.28	Slow Operations	5%					✓	✗	✗	✗	✓
		glibc-2.21	Slow Operations	7%					✓	✗	✗	✗	✓
	dedup	glibc-2.28	Slow Operations	6%					✓	✗	✗	✗	✓
		glibc-2.21	Kernel Contention	20%					✓	✗	✗	✗	
		jemalloc	Slow Operations	6%					✓	✗	✗	✗	✓
Slow Memory Accesses	swaptions	Hoard	Lock Contention	105%					✓	✓	✗	✗	✓
	cache-scratch	glibc-2.28	False Sharing	1007%	36	26	0	30	✓	✗	✗	✗	
		TCMalloc	False Sharing	3754%	3	89	0	75	✓	✗	✗	✗	
		Hoard	False Sharing	1943%	8	36	0	47	✓	✗	✗	✗	
	cache-thrash	TCMalloc	False Sharing	3788%	22	79	0	77	✓	✗	✗	✗	
	freqmine	glibc-2.21	Conflict Misses	6%	98	96	11	0	✓	✗	✗	✗	✓
		jemalloc	Conflict Misses	12%	93	99	14	0	✓	✗	N/A	✗	✓
		TCMalloc	Conflict Misses	11%	97	99	13	0	✓	✗	✗	✗	✓
	Quicksilver	glibc-2.28	Conflict Misses	55%	77	87	14	0	✓	✗	✗	✗	✓
	raytrace	glibc-2.28	Conflict Misses	27%	78	78	52	0	✓	✗	✗	✗	
		glibc-2.21	Conflict Misses	27%	74	75	60	0	✓	✗	✗	✗	✓
		jemalloc	Conflict Misses	20%	86	96	79	0	✓	✗	✗	✗	✓
		Hoard	Conflict Misses	10%	76	69	7	0	✓	✗	✗	✗	✓

has a known bug in invoking `madvise()` [Gorman 2015]. But for allocator-induced conflict misses, we did not find any allocator that could handle conflict misses. We think that it is difficult to have the allocator to detect conflict misses, which actually indicates the importance to have an allocator-profiler like MemPerf.

4.3 Compare with Existing Work and Case Studies

As mentioned in Section 4.2, MemPerf could precisely identify allocator-induced performance degradation. We further compared with existing profiling tools, including perf [Eranian et al. 2015], CCProf [Roy et al. 2018], and Feather [Chabbi et al. 2018]. Among them, perf [Eranian et al. 2015] is a general profile tool that could identify different kinds of performance issues relying on hardware performance counter, CCProf [Roy et al. 2018] is the state-of-art tool of detecting cache conflict misses, and Feather [Chabbi et al. 2018] is the state-of-art tool in identifying false sharing performance problems.

As shown in Table 4, MemPerf reports 18 issues for all evaluated applications, including 6 issues caused by slow memory management operations, and 12 issues due to slow memory accesses. In contrast, existing work could not identify the real root causes of most slowdowns (except swaptions). In particular, perf can not identify false sharing or conflict misses caused by the allocator, but only reports the statements with cache misses. For most slow memory management operations, perf cannot clearly indicate the performance issue of the allocator. For instance, canneal has 7% slowdown on glibc-2.21, but perf’s top five time-consuming functions are not related to the allocator. Although CCProf and Feather can report cache-related issues, but cannot attribute the issues to the allocator. Therefore, MemPerf has better effectiveness in identifying memory-induced performance issues.

In the following, we utilize five case studies to present the difference between MemPerf and the other tools.

Case Study 1 - dedup: When the allocator is libc-2.21, dedup runs 28% slower than the one with libc-2.28. CCProf and Feather could not detect this case. When displaying stack traces, perf reports 13.53% time in kernel's page fault handler, 8.38% time in madvise system call, and reports 5.12% time in malloc. Given such a report, it is hard to identify a performance issue inside, not to mention relating the issue with the memory allocator, since the stack trace of madvise shows nothing related to memory allocations/deallocations.

In contrast, MemPerf directly predicts the large performance slowdown and reports excessive madvise() inside free() [Gorman 2015], as shown in Figure 5. MemPerf reports that each free has around 0.3 madvise invocations, which is abnormally high, and the runtime of each madvise is about 11260 cycles. Based on such a report, we could understand that the allocator tries to release the memory back to the OS aggressively, which could further introduce high kernel contention between page faults and memory-related system calls. Since the OS will utilize the same lock to protect these operations, this is the reason why the runtime of each madvise is extremely high. As listed in Table 3, the allocator-induced kernel contention issue can be solved by switching the allocator or changing the configuration of the allocator. In this case, we could change the threshold of shrink_heap. Such a change improves the performance of dedup with libc-2.21 by 20%.

```
Large performance slowdown caused by slow memory management operations:
# 0 free() related with small-sized objects,
where each call takes 4302 cycles having 0.3 madvise(),
and each madvise() takes 11260 cycles on average
```

Fig. 5. MemPerf's output for dedup with libc-2.21.

Case Study 2 - swaptions: When the allocator is Hoard, swaptions runs 2× slower. For this case, perf can report 44.79% of the time spent in pthread_spin_lock, where 36.46% are caused by the Hoard allocator. MemPerf can report a high contention rate for this case, as shown in Figure 6. This issue can be solved by switching the allocator or changing Hoard's configuration, as shown in Table 3. MemPerf further pinpoints the contention mainly occurring inside medium-sized objects. Then we further identify that it is caused by an inappropriate threshold (in hoardThreshold FunctionClass) to control the returning of a super-block to the global pool. Unfortunately, one deallocation may move the current super-block to the global pool, but the next allocation will move the super-block back. Therefore, many threads may contend for the same lock protecting the global pool due to frequent back-and-forth movement. We have reported this bug to the authors of Hoard and got their confirmation. This bug could be reduced/fixed by simply changing two flags defined in include/hoard/hoardheap.h: changing EMPTINESS_CLASSES to 2 achieves 42% speedup, and changing SUPERBLOCK_SIZE from 64K to 4K achieves 105% performance improvement.

```
Large performance slowdown caused by slow memory management operations:
# 0 malloc() related with medium-sized objects,
where each call takes 4506 cycles with 1.7 locks (contention rate 34%)
# 1 free() related with medium-sized objects,
where each call takes 5871 cycles with 2.8 locks (contention rate 25%)
```

Fig. 6. MemPerf's output for swaptions with Hoard.

Case Study 3 - cache-scratch: Allocators may introduce false sharing on this application, where TCMalloc will introduce $2.7\times$ slowdown comparing to glibc-2.28. Similarly, existing tools cannot report the real root cause. perf only reports that 99.98% of the time is spent on worker, which is very normal for this simple program, and the lines with frequent cache misses. Feather reports false sharing inside worker, but cannot attribute the issue to the allocator. It also could not report the size information and callsite information. Instead, as shown in Figure 7, MemPerf directly pinpoints that cache-scratch suffers from allocator-induced false sharing by its high FSS score (i.e., 75), and reports the corresponding allocation callsites and sizes. As shown in Table 3, this issue can be fixed by using another allocator or changing the size of the corresponding objects. By simply changing the sizes to be 64 bytes, we improve the performance of cache-scratch with TCMalloc by $37.9\times$.

```
Allocator-induced false sharing, with FSS = 75
All related objects are allocated at cache-scratch.cpp:80, size 8 bytes
```

Fig. 7. MemPerf's output for cache-scratch with TCMalloc.

Case Study 4 - Quicksilver: The execution with glibc-2.28 runs around 46% slower than those with glibc-2.21, TCMalloc, and jemalloc. However, existing tools cannot report the real root cause. perf only reports that 31.66% of the time is spent on NuclearData::getReactionCrossSection and reports the program lines with cache misses, without reporting allocator-induced conflict misses. CCProf does not report any issues of conflict misses that affect the performance.

Instead, MemPerf directly pinpoints the allocator-induced conflict misses via high CMS score (i.e., 14), and reports the corresponding allocation callsites and sizes as Figure 8. As listed in Table 3, this issue can be fixed by using another allocator, adding padding to the corresponding heap objects, or inserting additional bogus objects. Based on the report, we add 64-byte padding to allocations from QS_Vector, a data structure widely used in Quicksilver, and the performance is improved by 55%.

```
Allocator-induced conflict misses, with CMS = 14
Information of related objects:
#0 Allocated at MemoryControl.hh:21, size 32, 248, 272, and 1840 bytes
.....
```

Fig. 8. MemPerf's output for Quicksilver with glibc-2.28

Case Study 5 - raytrace: Multiple allocators have allocator-induced conflict misses on this application. For instance, the execution with jemalloc runs 20% slower than that with TCMalloc. However, existing tools cannot correctly report the root cause of this issue. perf generates a report as shown in Figure 9. First, it reports that 29.51% of the time is spent in std::_Rb_tree. Then click on this function will generate a report as shown in the lower part that the instruction mov 0x18(%rdi),%rdi introduces 66.82% of cycles of std::_Rb_tree. However, such a behavior can be caused by multiple reasons, such as true/false sharing on the variable __x. It is difficult for users to link the slowdown to the conflict misses caused by the allocator directly. CCProf reports conflict misses in the loop at line 1871 of c++/7/bits/stl_tree.h. However, it cannot report where these objects are allocated, and whether such an issue is allocator-induced or not. Therefore, it is still

challenging for users to identify the allocator-induced issue and figure out why there are excessive cache misses inside the C++ STL library. Therefore, users cannot fix the performance issue based on its report.

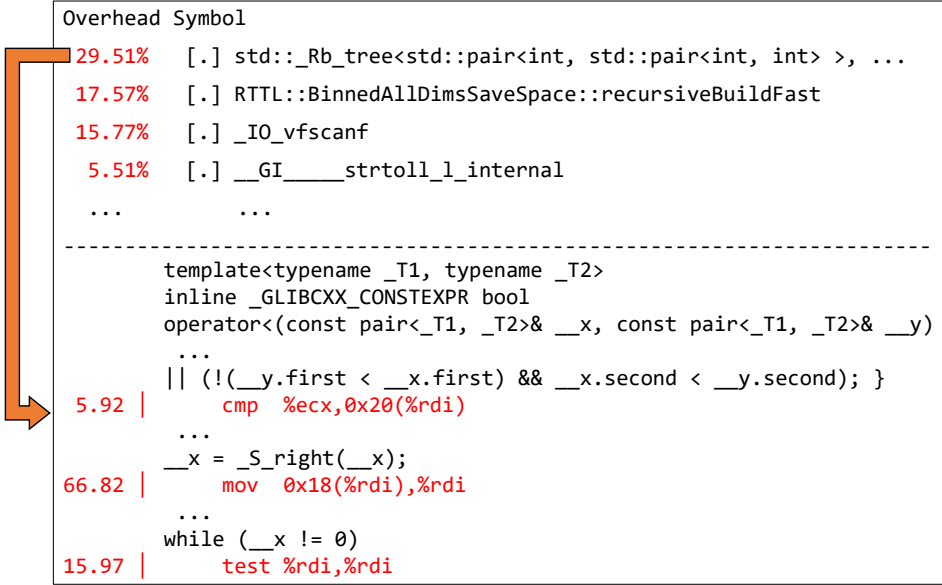


Fig. 9. perf's output for raytrace with jemalloc.

Instead, MemPerf not only pinpoints the allocator-induced conflict misses (with the conflict miss score to be 79), but also reports the detailed information of related heap objects, showing their allocation callsites and sizes as Figure 10. As listed in Table 3, such a bug can be fixed by switching to a new allocator, adding padding to the corresponding heap objects, or inserting additional bogus objects. Guided by the report, we could insert some bogus 48-byte allocations (without actually using them) between the reported sites, which improves the performance by 20%.

```
Allocator-induced conflict misses, with CMS = 79
Information of related objects:
#0 Allocated at /usr/include/c++/7/bits/stl_map.h:493, size 48 bytes
#1 Allocated at MiniView/ObjParser.hxx:449, size 48 bytes
.....
```

Fig. 10. MemPerf's output for raytrace with jemalloc.

4.4 Performance Overhead of MemPerf

The performance overhead of MemPerf can be seen in Figure 11, where all data is normalized to the data of the default allocator (glibc-2.28). Overall, MemPerf introduces around 7.5% performance overhead (GEOMEAN), while perf's overhead is 20%, CCProf's overhead is 171%, and Feather's overhead is 56% (including offline). Compared with the other three tools, MemPerf introduces the

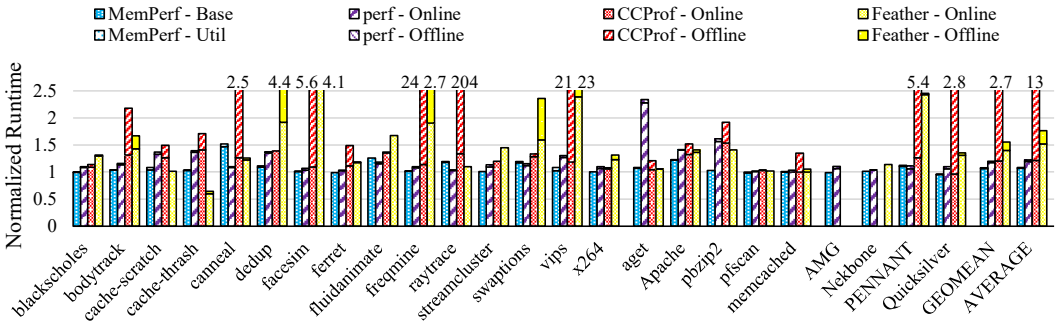


Fig. 11. Performance overhead of MemPerf, perf, CCProf [Roy et al. 2018], and Feather [Chabbi et al. 2018], normalized to the default Linux allocator. “MemPerf-Base” is the overhead without measuring page/cache utilization rate, and “MemPerf-Util” is the additional overhead with such measurements.

lowest overhead on average. Among that, MemPerf’s base implementation (without measuring page/utilization rate) is around 6.5%, while measuring page/cache utilization rate adds around 1% performance overhead.

For 20 out of 24 applications, except canneal, fluidanimate, swaptions, and apache, MemPerf imposes less than 20% overhead. MemPerf introduces 52% performance overhead for canneal, since canneal has an excessive number of memory management operations, around 1.3 million each second. Similarly, MemPerf introduces around 20% overhead for swaptions due to large numbers of allocations and deallocations. For each management operation, MemPerf should check a global hash table in order to differentiate a new/re-used object, saves the information of these memory objects, and then updates the used bytes of the corresponding cache lines and pages. For each memory operation, MemPerf also invokes RDTSCP twice to collect the runtime and updates multiple counters as well.

MemPerf imposes 26% overhead for fluidanimate due to the sampling mechanism. The hardware sampling introduces 12% overhead for fluidanimate, since it has more extensive sampled accesses than other applications. fluidanimate has 6,046 sampled accesses each second, but for the other applications in PARSEC, there are only 2,372 sampled accesses each second in average. MemPerf imposes 22% overhead for Apache, which we do not know the exact reason for its high overhead.

We also evaluate the MemPerf’s impact on cache/TLB misses of applications. Specifically, we compared the miss rates of load instructions on the cache (cache load miss rates), the miss rates of TLB on load instructions (TLB load miss rates), and the miss rates of TLB on store instructions (TLB store miss rates) in the PARSEC benchmarks with glibc-2.28 as the allocator, under two conditions: with and without the use of MemPerf. Due to perf’s limitations, we were unable to measure the miss rates of store instructions on the cache (cache store miss rates).

As seen from Table 5, the MemPerf’s impact on cache/TLB misses was generally minimal. Most of the changes were within approximately 0.5%, with the exception of canneal, which had a 5.05% reduction in cache load miss rates and a 2.31% reduction in TLB load miss rates when MemPerf was enabled. The reductions in canneal are due to MemPerf handling canneal’s extensive memory management operations. Specifically, the operations performed by MemPerf had lower miss rates than the default operations of canneal, leading to an overall reduction in miss rates. Overall, MemPerf induces minimal changes to cache/TLB miss rates under most circumstances, suggesting that MemPerf’s profiling effectively reflects the true behavior of the application.

Table 5. The impact of MemPerf on the miss rates of cache load instructions, TLB load instructions, and TLB store instructions for the PARSEC benchmarks. Each column represents the change in the respective miss rate (expressed as a percentage) with MemPerf enabled compared to without MemPerf. A positive value indicates an increase in the miss rate, and a negative value indicates a decrease.

Benchmark	Cache Load	TLB Load	TLB Store
blackscholes	0.01%	0.00%	0.01%
bodytrack	-0.05%	0.00%	0.01%
canneal	-5.05%	-2.31%	0.06%
dedup	0.20%	0.00%	0.02%
facesim	0.00%	0.00%	0.00%
ferret	-0.14%	0.00%	0.06%
fluidanimate	-0.23%	0.00%	0.03%
frequmine	0.01%	0.00%	0.00%
raytrace	-0.58%	0.00%	0.01%
streamcluster	0.00%	0.00%	0.10%
swaptions	0.10%	0.00%	0.00%
vips	0.04%	0.00%	0.04%
x264	0.00%	0.00%	0.01%

4.5 Design Choices

Thread-aware and Type-aware Performance Modeling: MemPerf proposes a thread-aware and type-aware performance modeling to identify slow management operations. In contrast, the naive algorithm is the one without considering the threads and different types of memory management operations. Basically, they will combine all runtimes of different threads together, and all runtimes of memory operations, without differentiating the type of operations. In fact, as shown in Table 2, the naive algorithm fails to report slow management operations of 5 executions of canneal and dedup, indicating the importance of the proposed modeling.

Sensitiveness To Sampling Rate: We further investigate the performance and effectiveness impacts under different sampling rates. By default, MemPerf samples one memory access out of 500,000 ones. The sensitiveness evaluation is based on cache-scratch, cache-thrash, and all PARSEC applications (15 applications in total). With the sampling rate of 5 million, 500 thousand (the default one), and 50 thousand, MemPerf's performance overheads are 9.2%, 9.5%, and 10.3% respectively. Note that the average overhead here is higher than those shown in Section 4.4, since we only include 15 applications. The effectiveness impact of different sampling rates is shown in Table 6. That is, when using a sampling rate of 5 million, MemPerf could miss four cases of raytrace and two cases of frequmine. We also confirmed that using the sampling rate of 50,000 does not discover more issues. Overall, the default sampling rate provides a good balance between performance and effectiveness.

5 RELATED WORK

Memory Allocation Profiling: Many allocator profilers focus on memory usage. Mprof [Zorn and Hilfinger 1988] collects the information about allocations and deallocations, reports memory leaks, an allocation bin table, and a direct allocation table that shows the memory usage of functions. Mtrace [Free Software Foundation, Inc. 2015] and Mtrace++ [Lee et al. 2000] trace the allocation events and report possible memory leaks. Mtrace++ further identifies the origins of allocated memory and the life spans of objects. TcMalloc's profiler reports the heap state dynamically, locates memory leaks, and identifies statements with a large number of allocations [Ghemawat and Menage 2007]. Chang et. al. profile allocation behavior of Android applications [Chang et al.

Table 6. The impact of MemPerf's effectiveness under the sampling rate of 5 million, 500 thousand, and 50 thousand.

Application	Allocator	5 Million	500 Thousand	50 Thousand
cache-scratch	glibc-2.28	✓	✓	✓
	TCMalloc	✓	✓	✓
	Hoard	✓	✓	✓
cache-thrash	TCMalloc	✓	✓	✓
	glibc-2.21	✗	✓	✓
	jemalloc	✗	✓	✓
freqmine	TCMalloc	✓	✓	✓
	glibc-2.28	✗	✓	✓
	glibc-2.21	✗	✓	✓
raytrace	jemalloc	✗	✓	✓
	Hoard	✗	✓	✓

2014]. MemoryProfiler shows memory consumption of different types (e.g., Java, Native) and different objects, timeline information of allocations, garbage collection events, and the stack trace of each allocation inside Android applications [Android Community 2020]. WMTrace monitors allocations and deallocations, together with their timing information [Perks et al. 2011]. WMTrace also compresses the trace and writes it to a persistent file. LeakPoint detects memory leaks of different size classes [Clause and Orso 2010]. However, they mainly focus on memory consumption and memory leaks of applications. They cannot identify the slowdowns caused by allocators and design issues within the allocator, which is significantly different from MemPerf.

General Profilers: There exist many general profilers that could help identify the performance issues of applications, such as gprof [Graham et al. 1982], Oprofile [Levon and Elie 2004], Coz [Curtsinger and Berger 2015], and Perf [Eranian et al. 2015]. As discussed in Section 1, they typically focus on performance issues of applications, providing no insights into the allocators.

Tools of Detecting Cache Misses: Many profilers focus on cache misses in applications. Some tools could detect capacity misses [Liu et al. 2014; Lucia [n. d.]; Pesterev et al. 2010], some focus on cache conflict misses [Pesterev et al. 2010; Roy et al. 2018], and others detect false/true sharing [Chabbi et al. 2018; Khan et al. 2019; Liu and Berger 2011; Liu and Liu 2016; Liu et al. 2014; Luo et al. 2016; Pesterev et al. 2010; Zhao et al. 2011]. However, none of the existing work can attribute cache misses to the allocator, which still requires significant effort to pinpoint the real root causes of allocator-induced performance degradation.

Study of Memory Allocators: There exist multiple studies that investigate different memory allocators [Barroso et al. 1998; Chihaiia and Gross 2004; Ferreira et al. 2011; Matias et al. 2011; Zavrtanik and Mihelic [n. d.]]. However, they can only report some observable behavior of evaluated allocators, providing no insight into the specific issues inside of the allocators. They could not report memory overhead caused by an allocator as well.

6 CONCLUSION

None of the existing profilers help understand the slowdown caused by an allocator. This paper designs the first general profiler to profile the performance impacts of a memory allocator, through the use of simple counters, timestamp registers, and hardware Performance Monitoring Units. It has the potential to benefit both normal users and allocator designers. Our extensive evaluation has confirmed that MemPerf identifies most performance slowdowns (at 98%) caused by allocators, and helps identify multiple design issues in widely-used allocators. Therefore, MemPerf is an indispensable complement to existing profilers.

ACKNOWLEDGMENTS

We thank our anonymous reviewers for their helpful comments on improving this paper. This material is based upon work supported by the National Science Foundation under Awards DUE-2215193, CCF-2024253, and CNS-1750760. Any opinions, findings, and conclusions or recommendations expressed in this material are those of the author(s) and do not necessarily reflect the views of the National Science Foundation.

REFERENCES

- Martin Aigner, Christoph M. Kirsch, Michael Lippautz, and Ana Sokolova. 2015. Fast, multicore-scalable, low-fragmentation memory allocation through large virtual memory and global data structures. In *Proceedings of the 2015 ACM SIGPLAN International Conference on Object-Oriented Programming, Systems, Languages, and Applications, OOPSLA 2015, part of SPLASH 2015, Pittsburgh, PA, USA, October 25-30, 2015*. 451–469. <https://doi.org/10.1145/2814270.2814294>
- Mohammad Mejbah ul Alam, Tongping Liu, Guangming Zeng, and Abdullah Muzahid. 2017. SyncPerf: Categorizing, Detecting, and Diagnosing Synchronization Performance Bugs. In *Proceedings of the Twelfth European Conference on Computer Systems (Belgrade, Serbia) (EuroSys '17)*. 298–313. <https://doi.org/10.1145/3064176.3064186>
- Android Community. 2020. View the Java heap and memory allocations with citep. <https://developer.android.com/studio/profile/memory-profiler>.
- The OpenMP ARB. 2022. The OpenMP API Specification For Parallel Programming. <https://www.openmp.org/>.
- L.A. Barroso, K. Gharachorloo, and E. Bugnion. 1998. Memory system characterization of commercial workloads. In *Proceedings. 25th Annual International Symposium on Computer Architecture (Cat. No.98CB36235)*. 3–14. <https://doi.org/10.1109/ISCA.1998.694758>
- Emery D. Berger, Kathryn S. McKinley, Robert D. Blumofe, and Paul R. Wilson. 2000. Hoard: A Scalable Memory Allocator for Multithreaded Applications. In *Proceedings of the Ninth International Conference on Architectural Support for Programming Languages and Operating Systems (Cambridge, Massachusetts, USA) (ASPLOS IX)*. 117–128. <https://doi.org/10.1145/378993.379232>
- Christian Bienia and Kai Li. 2009. PARSEC 2.0: A New Benchmark Suite for Chip-Multiprocessors. <http://www.mount.ece.umn.edu/~jjyi/MoBS/2009/program/02E-Bienia.pdf>.
- Milind Chabbi, Shasha Wen, and Xu Liu. 2018. Featherlight On-the-Fly False-Sharing Detection. In *Proceedings of the 23rd ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming (Vienna, Austria) (PPoPP '18)*. 152–167. <https://doi.org/10.1145/3178487.3178499>
- K. Chang, C. Kao, Y. Chen, and G. Chen. 2014. Memory behavior profiler for Android applications. In *2014 IEEE 3rd Global Conference on Consumer Electronics (GCCE)*. 634–635. <https://doi.org/10.1109/GCCE.2014.7031343>
- I. Chihai and T. Gross. 2004. Effectiveness of simple memory models for performance prediction. In *Performance Analysis of Systems and Software, 2004 IEEE International Symposium on - ISPASS*. 98–105. <https://doi.org/10.1109/ISPASS.2004.1291361>
- James Clause and Alessandro Orso. 2010. LEAKPOINT: Pinpointing the Causes of Memory Leaks. In *Proceedings of the 32Nd ACM/IEEE International Conference on Software Engineering - Volume 1 (Cape Town, South Africa) (ICSE '10)*. 515–524. <https://doi.org/10.1145/1806799.1806874>
- Jon Coppeard. 2017. Allocate all JS data in a separate jemalloc arena. https://bugzilla.mozilla.org/show_bug.cgi?id=1410132.
- Charlie Curtsinger and Emery D. Berger. 2015. Coz: Finding Code That Counts with Causal Profiling. In *Proceedings of the 25th Symposium on Operating Systems Principles (Monterey, California) (SOSP '15)*. 184–197. <https://doi.org/10.1145/2815400.2815409>
- Stephane Eranian, Eric Gouriou, Tipp Moseley, and Willem de Bruijn. 2015. Linux kernel profiling with perf. <https://perf.wiki.kernel.org/index.php/Tutorial>.
- Jason Evans. 2016. Scalable memory allocation using jemalloc. <https://krebsonsecurity.com/2016/10/ddos-on-dyn-impacts-twitter-spotify-reddit/>.
- Tais B. Ferreira, Rivalino Matias, Autran Macêdo, and Lucio B. Araujo. 2011. A Comparison of Memory Allocators for Multicore and Multithread Applications: A Quantitative Approach. In *2011 Brazilian Symposium on Computing System Engineering*. 200–205. <https://doi.org/10.1109/SBESC.2011.29>
- Free Software Foundation, Inc. 2015. The GNU C Library: Allocation Debugging. http://www.gnu.org/software/libc/manual/html_node/Allocation-Debugging.html.
- Sanjay Ghemawat. 2005. Profiling heap usage. http://goog-perftools.sourceforge.net/doc/heap_profiler.html.
- Sanjay Ghemawat and Paul Menage. 2007. TCMalloc: Thread-caching malloc, 2007. <http://goog-perftools.sourceforge.net/doc/tcmalloc.html>.
- Mel Gorman. 2015. malloc: Reduce worst-case behaviour with madvise and refault overhead. <https://patchwork.ozlabs.org/project/glibc/patch/20150209140608.GD2395@suse.de/>.

- Susan L. Graham, Peter B. Kessler, and Marshall K. McKusick. 1982. gprof: a Call Graph Execution Profiler. In *SIGPLAN Symposium on Compiler Construction*. 120–126. <https://doi.org/10.1145/872726.806987>
- Intel Corporation. 1997. Using the rdtsc instruction for performance monitoring. <https://www.ccsf.carleton.ca/~jamuir/rdtscpm1.pdf>. *Techn. Ber., Tech. Rep., Intel Corporation* (1997), 22.
- Tanvir Ahmed Khan, Yifan Zhao, Gilles Pokam, Barzan Mozafari, and Baris Kasikci. 2019. Huron: Hybrid False Sharing Detection and Repair. In *Proceedings of the 40th ACM SIGPLAN Conference on Programming Language Design and Implementation* (Phoenix, AZ, USA) (*PLDI 2019*). Association for Computing Machinery, New York, NY, USA, 453–468. <https://doi.org/10.1145/3314221.3314644>
- Bradley C. Kuszmaul. 2015. SuperMalloc: a super fast multithreaded malloc for 64-bit machines. In *Proceedings of the 2015 International Symposium on Memory Management* (Portland, OR, USA) (*ISMM '15*). Association for Computing Machinery, New York, NY, USA, 41–55. <https://doi.org/10.1145/2754169.2754178>
- Lawrence Livermore National Laboratory. 2018. CORAL-2 Benchmarks. <https://asc.llnl.gov/coral-2-benchmarks>.
- Woo Hyong Lee, J. Morris Chang, and Yusuf Hasan. 2000. A Dynamic Memory Measuring Tool for C++ Programs. In *Proceedings of the 3rd IEEE Symposium on Application-Specific Systems and Software Engineering Technology (ASSET'00)* (*ASSET '00*). IEEE Computer Society, Washington, DC, USA, 155–. <https://doi.org/10.1109/ASSET.2000.888070>
- Daan Leijen. 2020. mimalloc. <https://github.com/microsoft/mimalloc>.
- John Levon and Philippe Elie. 2004. Oprofile: A system profiler for linux. <https://oprofile.sourceforge.io/news/>.
- Tongping Liu and Emery D. Berger. 2011. SHERIFF: precise detection and automatic mitigation of false sharing. In *Proceedings of the 2011 ACM international conference on Object oriented programming systems languages and applications* (Portland, Oregon, USA) (*OOPSLA '11*). 3–18. <https://doi.org/10.1145/2048066.2048070>
- Tongping Liu and Xu Liu. 2016. Cheetah: Detecting False Sharing Efficiently and Effectively. In *Proceedings of the 2016 International Symposium on Code Generation and Optimization* (Barcelona, Spain) (*CGO 2016*). 1–11. <https://doi.org/10.1145/2854038.2854039>
- Tongping Liu, Chen Tian, Ziang Hu, and Emery D. Berger. 2014. PREDATOR: Predictive False Sharing Detection, Vol. 49. Association for Computing Machinery, New York, NY, USA. <https://doi.org/10.1145/2692916.2555244>
- Xu Liu and John Mellor-Crummey. 2013. A Data-Centric Profiler for Parallel Programs. Association for Computing Machinery, New York, NY, USA. <https://doi.org/10.1145/2503210.2503297>
- Xu Liu, Kamal Sharma, and John Mellor-Crummey. 2014. ArrayTool: A Lightweight Profiler to Guide Array Regrouping. In *Proceedings of the 23rd International Conference on Parallel Architectures and Compilation* (Edmonton, AB, Canada) (*PACT '14*). 405–416. <https://doi.org/10.1145/2628071.2628102>
- X. Liu, K. Sharma, and J. Mellor-Crummey. 2014. ArrayTool: A lightweight profiler to guide array regrouping. In *2014 23rd International Conference on Parallel Architecture and Compilation Techniques (PACT)*. 405–415. <https://doi.org/10.1145/2628071.2628102>
- Brandon Lucia. [n. d.]. MultiCacheSim: A coherent multiprocessor cache simulator. <https://courses.cs.washington.edu/courses/cse471/11sp/sim.html>.
- Chi-Keung Luk, Robert Cohn, Robert Muth, Harish Patil, Artur Klauser, Geoff Lowney, Steven Wallace, Vijay Janapa Reddi, and Kim Hazelwood. 2005. Pin: Building Customized Program Analysis Tools with Dynamic Instrumentation. In *Proceedings of the 2005 ACM SIGPLAN Conference on Programming Language Design and Implementation* (Chicago, IL, USA) (*PLDI '05*). ACM, New York, NY, USA, 190–200. <https://doi.org/10.1145/1065010.1065034>
- Liang Luo, Akshitha Sriraman, Brooke Fugate, Shiliang Hu, Gilles Pokam, Chris J. Newburn, and Joseph Devietti. 2016. LASER: Light, Accurate Sharing dEtection and Repair. In *2016 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. 261–273. <https://doi.org/10.1109/HPCA.2016.7446070>
- Adrian LUPASC and Viorica Popoiu. 2014. Dynamic Memory Allocation–Clr Profiler. , 118 pages.
- R. Matias, T. Borges Ferreira, and A. Macedo. 2011. An experimental study on user-level memory allocators in middleware applications. In *2011 IEEE International Conference on Systems, Man, and Cybernetics*. 2431–2436. <https://doi.org/10.1109/ICSMC.2011.6084042>
- Oliver Perks, Simon D. Hammond, Simon J. Pennycook, and Stephen A. Jarvis. 2011. WMTools - Assessing Parallel Application Memory Utilisation at Scale. In *Proceedings of the 8th European Conference on Computer Performance Engineering* (Borrowdale, UK) (*EPEW'11*). Springer-Verlag, Berlin, Heidelberg, 148–162. https://doi.org/10.1007/978-3-642-24749-1_12
- Aleksey Pesterev, Nickolai Zeldovich, and Robert T. Morris. 2010. Locating Cache Performance Bottlenecks Using Data Profiling. In *Proceedings of the 5th European Conference on Computer Systems* (Paris, France) (*EuroSys '10*). 335–348. <https://doi.org/10.1145/1755913.1755947>
- The Open MPI Project. 2022. Open MPI: Open Source High Performance Computing. <https://www.open-mpi.org/>.
- Kirill Rogozhin. 2014. Controlling memory consumption with Intel® Threading Building Blocks (Intel® TBB) scalable allocator. <https://software.intel.com/content/www/us/en/develop/articles/controlling-memory-consumption-with-intel-threading-building-blocks-intel-tbb-scalable.html>.

- Probir Roy, Shuaiwen Leon Song, Sriram Krishnamoorthy, and Xu Liu. 2018. Lightweight Detection of Cache Conflicts. In *Proceedings of the 2018 International Symposium on Code Generation and Optimization* (Vienna, Austria) (CGO 2018). Association for Computing Machinery, New York, NY, USA, 200–213. <https://doi.org/10.1145/3168819>
- Konstantin Serebryany, Derek Bruening, Alexander Potapenko, and Dmitry Vyukov. 2012. AddressSanitizer: a fast address sanity checker. In *Proceedings of the 2012 USENIX conference on Annual Technical Conference* (Boston, MA) (USENIX ATC'12). USENIX Association, Berkeley, CA, USA, 28–28. <http://dl.acm.org/citation.cfm?id=2342821.2342849>
- Tianwei Sheng, Neil Vachharajani, Stephane Eranian, Robert Hundt, Wenguang Chen, and Weimin Zheng. 2011. RACEZ: A Lightweight and Non-invasive Race Detection Tool for Production Applications. In *Proceedings of the 33rd International Conference on Software Engineering* (Waikiki, Honolulu, HI, USA) (ICSE '11). 401–410. <https://doi.org/10.1145/1985793.1985848>
- Oliver Yang. 2015. Pitfalls of TSC usage. <http://oliveryang.net/2015/09/pitfalls-of-TSC-usage/>.
- Tingting Yu and Michael Pradel. 2016. SyncProf: detecting, localizing, and optimizing synchronization bottlenecks. In *Proceedings of the 25th International Symposium on Software Testing and Analysis, ISSTA 2016, Saarbrücken, Germany, July 18-20, 2016*, Andreas Zeller and Abhik Roychoudhury (Eds.). 389–400. <https://doi.org/10.1145/2931037.2931070>
- Matej Zavrtanik and Jurij Mihelic. [n. d.]. Experimental Evaluation and Comparison of Memory Allocators in the GNU/Linux Operating System. <http://ipsitranactions.org/journals/papers/tar/2017jan/p10.pdf>.
- Qin Zhao, David Koh, Syed Raza, Derek Bruening, Weng-Fai Wong, and Saman Amarasinghe. 2011. Dynamic Cache Contention Detection in Multi-Threaded Applications. In *Proceedings of the 7th ACM SIGPLAN/SIGOPS International Conference on Virtual Execution Environments* (Newport Beach, California, USA) (VEE'11). Association for Computing Machinery, New York, NY, USA, 27–38. <https://doi.org/10.1145/1952682.1952688>
- Fang Zhou, Yifan Gan, Sixiang Ma, and Yang Wang. 2018. wPerf: Generic Off-CPU Analysis to Identify Bottleneck Waiting Events. In *13th USENIX Symposium on Operating Systems Design and Implementation, OSDI 2018, Carlsbad, CA, USA, October 8-10, 2018*, Andrea C. Arpaci-Dusseau and Geoff Voelker (Eds.). USENIX Association, 527–543. <https://www.usenix.org/conference/osdi18/presentation/zhou>
- Benjamin Zorn and Paul N. Hilfinger. 1988. *A Memory Allocation Profiler for C and Lisp Programs*. Technical Report UCB/CSD-88-404. EECS Department, University of California, Berkeley. <http://www2.eecs.berkeley.edu/Pubs/TechRpts/1988/5382.html>

Received 2023-04-14; accepted 2023-08-27